

Documentacion Tecnica

UniBot / Tutor-unis — Tutor IA para Medicarama

Documento de traspaso (handover) para desarrollo

Chatbot educativo con RAG filtrado por curso (cafeo) sobre Google Cloud: FastAPI + Gemini 2.5 Flash + Vertex AI Search + Firebase. Este documento describe la arquitectura completa, cada fichero, los endpoints, la seguridad, el despliegue y la deuda tecnica para que otra persona pueda retomar el proyecto sin contexto previo.

Proyecto GCP: tutor-unis | Region: us-central1 | Servicio: unibot-medicarama
Fecha: 8 de julio de 2026

Indice

1. Resumen ejecutivo
2. Puesta en marcha rapida (onboarding)
3. Arquitectura general
4. Stack tecnologico y dependencias
5. Estructura del repositorio
6. Configuracion y variables de entorno
7. Autenticacion, registro y aislamiento de usuario
8. El capeo por curso (nucleo del sistema)
9. Modelos de IA, grounding y generacion de imagen
10. Endpoints del backend (app/main.py)
11. Memoria de conversacion
12. Tope de consumo diario por alumno
13. Frontend
14. Seguridad y privacidad
15. Evaluacion empirica (OE6)
16. Tests y utilidades
17. Despliegue (Cloud Run)
18. Operativa y resolucion de problemas
19. Estado del repositorio y deuda tecnica
20. Glosario

1. Resumen ejecutivo

UniBot (nombre interno del repositorio: **Tutor-unis**; nombre de producto: **Tutor Virtual de Medicarama**) es un asistente educativo conversacional para estudiantes de ciencias de la salud de la academia **Medicarama**. Responde dudas apoyandose EXCLUSIVAMENTE en el material oficial de los cursos que cada alumno ha comprado (RAG con filtrado por curso, el llamado "capeo"), genera tests autoevaluables, resuelve casos clinicos con metodo socratico, analiza documentos e imagenes que sube el alumno, genera ilustraciones medicas y exporta apuntes en PDF.

Es el proyecto de un **TFG**. Esta desplegado en produccion sobre Google Cloud (Cloud Run) y ya lo usan alumnos reales. Esta documentacion es un documento de **traspaso (handover)**: describe con precision la arquitectura, cada fichero, cada endpoint, la seguridad, el despliegue y la deuda tecnica, de modo que otra persona pueda retomar el desarrollo sin conocimiento previo.

1.1. Idea central en una frase

Un chatbot Gemini + RAG donde CADA alumno solo puede "ver" el material de los cursos que ha pagado. El filtrado (capeo) se hace en la capa de recuperacion (Vertex AI Search filtrando por `course_id`) y el modelo de lenguaje NO tiene herramienta de busqueda propia sobre ese corpus, de modo que es imposible que "se salte" el filtro. El sistema es **fail-closed**: ante la duda, no muestra material.

1.2. Datos clave del proyecto

Concepto	Valor
Proyecto GCP	tutor-unis
Region	us-central1
Servicio Cloud Run	unibot-medicarama
Modelo de lenguaje	Gemini 2.5 Flash (Vertex AI)
Modelo de imagen	Imagen 3.0 (imagen-3.0-generate-001) via SDK google-genai
RAG / buscador	Vertex AI Search (Discovery Engine), engine Enterprise 'medicarama-search'
Auth	Firebase Authentication (email/password + verificacion de email)
Base de datos	Cloud Firestore (coleccion 'users')
Almacenamiento	Google Cloud Storage (bucket privado + URLs firmadas)
Backend	FastAPI + Uvicorn (Python 3.10 en Docker)
Frontend	HTML + CSS + JavaScript vanilla (sin framework)
Origen de matriculas	Zoho -> Cloud Function 'registerNewUser' -> Firestore

2. Puesta en marcha rapida (onboarding)

Para levantar el proyecto en local y entender el flujo en ~30 minutos:

2.1. Requisitos previos

- Python 3.10+ y pip.
- Google Cloud CLI (gcloud) instalado.
- Acceso concedido al proyecto GCP tutor-unis (pide al responsable que te anada como principal con los roles de Vertex AI, Discovery Engine, Firestore y Storage).
- El fichero .env (NO esta en git; pidelo al responsable o reconstruyelo con la seccion 6 de este documento).

2.2. Instalacion y arranque local

```
# 1. Entorno virtual e instalacion de dependencias
python3 -m venv venv
source venv/bin/activate          # Windows: .\venv\Scripts\activate
pip install -r requirements.txt

# 2. Autenticacion con Google Cloud (Application Default Credentials).
#   Abre el navegador; inicia sesion con el correo autorizado en tutor-unis.
gcloud auth application-default login

# 3. Arrancar el servidor de desarrollo (recarga en caliente)
uvicorn app.main:app --reload
#   -> http://127.0.0.1:8000
```

Aviso importante sobre la autenticacion: la organizacion (mindic.com) fuerza reautenticaciones frecuentes. Es habitual ver el error 503 ... Reauthentication is needed. La solucion es volver a ejecutar `gcloud auth application-default login`. No es un bug del codigo.

2.3. Comprobaciones rapidas (smoke tests)

```
# Verifica que el caqueo por curso funciona de punta a punta (sin uvicorn):
python3 test_capeo_e2e.py "que es el dolor neuropatico"

# Verifica el path multimodal (imagen + material capado):
python3 test_multimodal_capeo.py

# Lista los ultimos usuarios y si traen 'cursos' desde Zoho:
python3 check_new_user.py

# Prueba la busqueda RAG filtrada por curso directamente:
python3 -m app.retrieval "dolor neuropatico" 3287 4139
```

3. Arquitectura general

El sistema tiene tres planos: (1) un **frontend** estatico servido por el propio backend, (2) un **backend FastAPI** que orquesta autenticacion, recuperacion capada y llamada a los modelos, y (3) los **servicios gestionados de Google Cloud** (Vertex AI, Vertex AI Search, Firebase Auth, Firestore, Storage). El registro de alumnos entra por un flujo aparte (Zoho -> Cloud Function) que NO vive en este repositorio.

3.1. Diagrama de componentes (flujo de una pregunta de texto)

```
NAVEGADOR (index.html + main.js + auth.js)
|
| 1) Firebase Auth: login + verificacion email -> ID token (JWT)
| 2) POST /chat (FormData: texto, archivos[])
|    Header: Authorization: Bearer <ID token>
|
v
CLOUD RUN -> FastAPI (app/main.py)
|
| 3) verify_firebase_token() --(Firebase Admin)--> valida JWT + email
| 4) get_user_cursos(uid)    --(Firestore)-----> course_ids comprados
| 5) buscar_contexto(q, ids)  --(Vertex AI Search)-> contexto FILTRADO
|    filter = course_id: ANY("3287","4139")      + fuentes (PDF+pag)
| 6) firmar_url(gs://...)    --(Storage/IAM)-----> URL firmada 15 min
| 7) model_capeo.generate_content(prompt + contexto inyectado, stream)
|    (Gemini 2.5 Flash SIN grounding -> no puede saltarse el capeo)
|
v
| 8) Respuesta en streaming NDJSON ({type:chunk}...{type:done,sources})
v
NAVEGADOR -> pinta con efecto maquina de escribir + fuentes clicables
```

3.2. Principio de diseno mas importante: el capeo fail-closed

El material de todos los cursos vive en un mismo corpus de Vertex AI Search, pero cada documento esta etiquetado con su `course_id`. La recuperacion SIEMPRE filtra por los `course_id` que el alumno ha comprado. El modelo que responde (`model_capeo`) NO lleva herramienta de grounding sobre ese corpus: recibe el contexto ya filtrado inyectado en el prompt. Asi, aunque el modelo "quisiera", no puede recuperar por su cuenta material de un curso no comprado. Si el alumno no tiene cursos, no se busca nada (cadena cerrada por defecto).

4. Stack tecnologico y dependencias

4.1. Dependencias de Python (requirements.txt)

Paquete	Para que se usa
fastapi / uvicorn	Servidor web ASGI y framework de endpoints.
python-multipart	Recibir formularios multipart (subida de archivos en /chat).
google-cloud-aiplatform	SDK Vertex AI: modelos Gemini (vertexai.generative_models).
google-generativeai	SDK NUEVO de Google para generar imagenes con Imagen (ver seccion 9).
google-cloud-discoveryengine	Cliente de Vertex AI Search (el capeo por curso).
google-cloud-storage	Firmar URLs de los PDFs del corpus (bucket privado).
reportlab	Generar los PDFs de apuntes y ESTA documentacion.
pydantic	Modelos de datos de entrada (app/models.py).
python-dotenv	Cargar el .env en local.
pillow	Soporte de imagenes (dependencia de reportlab / procesado).
jinja2	Plantillas (dependencia de FastAPI para respuestas).
firebase-admin	Verificar ID tokens de Firebase y leer/escribir Firestore.

4.2. Servicios de Google Cloud implicados

- **Vertex AI** (Gemini 2.5 Flash): razonamiento y generacion de texto/vision.
- **Imagen 3.0** via google-generativeai: generacion de ilustraciones medicas.
- **Vertex AI Search / Discovery Engine**: motor RAG con el engine Enterprise medicarama-search (necesario para 'extractive answers' como contexto).
- **Firebase Authentication**: registro/login por email y verificacion.
- **Cloud Firestore**: coleccion users (cursos comprados, contador de tokens).
- **Cloud Storage**: bucket privado con los PDFs; se sirven con URL firmada V4.
- **Cloud Run**: hosting del contenedor. **Cloud Build**: build desde codigo.
- **Cloud Functions** (fuera de este repo): registerNewUser (alta desde Zoho) y recordActiveSession (marca sesion activa).

5. Estructura del repositorio

```
tutor-unis/
|-- app/                                BACKEND (paquete Python)
|   |-- __init__.py                    Vacio (marca el paquete)
|   |-- main.py                        FastAPI: endpoints, 3 modos de /chat, streaming, imagen
|   |-- services.py                    Init Vertex AI, definicion de los 3 modelos, PDF, fuentes
|   |-- retrieval.py                   CAPEO: Firestore + Vertex AI Search filtrado + URLs firmadas + tokens
|   |-- config.py                      .env, system prompt, filtros de seguridad, limites
|   \-- models.py                      Modelos Pydantic (MemoryUpdate)
|-- static/                             FRONTEND estatico
|   |-- css/style.css                  Estilos
|   |-- js/auth.js                     Firebase Auth (login, registro, verificacion, reset)
|   |-- js/main.js                     Chat, streaming NDJSON, TTS/voz, PDF, historial local
|   \-- images/                        logo.png, iconos de audio, etc.
|-- index.html                          Shell HTML (login overlay + chat)
|-- evaluacion/                         Bateria de evaluacion (OE6): harness + CSVs + resultados
|   |-- harness_eval.py                Script de evaluacion (4 modos)
|   |-- preguntas*.csv                 Baterias de preguntas (entrada)
|   \-- resultados*.csv                Resultados (salida)
|-- docs/                              Esta documentacion (PDF + generador)
|-- test_capeo_e2e.py                   Smoke test end-to-end del capeo (texto)
|-- test_multimodal_capeo.py            Smoke test del path multimodal
|-- check_new_user.py                  Utilidad: ver altas recientes y sus cursos
|-- Dockerfile                         Imagen de contenedor para Cloud Run
|-- requirements.txt                    Dependencias Python
|-- .env                               Config y IDs (NO en git; ver seccion 6)
|-- .gitignore                         Excluye .env, __pycache__, venv...
|-- .gcloudignore                      Excluye del build de Cloud Run (.venv, .git, tests, docs, pdf...)
\-- README.txt                          Guia breve de arranque
```

6. Configuración y variables de entorno

La configuración se reparte entre el fichero `.env` (IDs de infraestructura, fuera de git) y `app/config.py` (prompt del sistema, filtros de seguridad y límites de generación, en git). El frontend tiene además la config PÚBLICA de Firebase incrustada en `static/js/auth.js` (clave de API de cliente y URL de la Cloud Function de registro; no son secretos).

6.1. Variables del fichero `.env`

Variable	Valor actual	Uso
PROJECT_ID	tutor-unis	Proyecto GCP.
REGION	us-central1	Region de Vertex AI / Cloud Run.
DATA_STORE_ID	tutor-unis_1765964560922	Data store del grounding CLÁSICO (path multimodal / PDF), SIN capeo.
DATA_STORE_ID_TFG	documentos-tfg_1767008483624	Data store de transcripciones/defensa de TFG (grounding abierto).
SEARCH_ENGINE_ID	medicarama-search	Engine Enterprise del CAPEO (retrieval.py). Sobre el store con <code>course_id</code> indexable.
DATA_STORE_LOCATION	global	Localización de los data stores.
OPEN_DATA_STORE_ID	(comentada)	Documentos ABIERTOS para todos los alumnos. Al definirla, el backend activa la herramienta de abiertos automáticamente. Hoy inactiva.

6.2. `app/config.py`

- **filtros_seguridad:** las 4 categorías de dano de Vertex a `BLOCK_ONLY_HIGH`. Es deliberado: en contexto médico con alumnos autenticados no se debe bloquear contenido clínico legítimo (farmacos, dosis, urgencias, salud sexual). `ONLY_HIGH` sigue cortando lo realmente dañino.
- **configuracion_tutor:** `temperature 0.3`, `max_output_tokens 16384`, `top_p 0.95`, `top_k 40`, `stop_sequences ['\nUsuario:', 'User:']`.
- **LIMITE_TOKENS_DIA = 400000:** tope diario de tokens (prompt+respuesta) por alumno (ver sección 12).
- **system_instruction:** el prompt maestro. Define el rol (UniBot, tutor sanitario), las reglas de oro (basarse solo en el material; NO citar fuentes en línea porque se muestran aparte) y los 3 modos de interacción: ESTUDIO (socrático), EXAMEN/QUIZ (soluciones ocultas entre `||...||`), y PROBLEMAS/CASOS (paso a paso, sin dar la solución salvo que la pidan).

El system prompt incluye una regla especial para IMÁGENES GENERADAS: si el alumno pregunta por la enfermedad de una ilustración que se ha generado, el modelo NO debe decir que no puede ver imágenes, sino proponer y explicar una enfermedad concreta y relevante del material del curso (ver sección 9).

7. Autenticacion, registro y aislamiento de usuario

7.1. Verificacion en el backend (app/main.py)

Todos los endpoints sensibles dependen de `verify_firebase_token`, que se inyecta con `Depends`. Lee el header `Authorization: Bearer <token>`, valida el ID token con Firebase Admin y exige que el email este verificado:

```
async def verify_firebase_token(request: Request) -> dict:
    auth_header = request.headers.get("Authorization", "")
    if not auth_header.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="missing_token")
    token = auth_header.split(" ", 1)[1].strip()
    decoded = firebase_auth.verify_id_token(token) # firma, caducidad, emisor
    if not decoded.get("email_verified", False):
        raise HTTPException(status_code=403, detail="email_not_verified")
    return decoded # claims: uid, email, email_verified...
```

- **401** `missing_token` / `empty_token` / `token_expired` / `token_revoked` / `invalid_token`: problemas con el JWT.
- **403** `email_not_verified`: token valido pero email sin confirmar.
- El `uid` de las claims es la CLAVE de todo: identifica al alumno en Firestore (sus cursos y su contador de tokens) y en la memoria de conversacion.

7.2. Flujo de registro/login (static/js/auth.js)

El frontend usa el SDK de Firebase Auth (modo email/password, persistencia local). `onAuthStateChanged` es la unica fuente de verdad del estado de sesion. La logica de alta es "login o registro":

- El alumno introduce email + contrasena y pulsa Entrar.
- Si el login falla con `user-not-found` / `invalid-credential`, se intenta un **registro** llamando por POST a la Cloud Function `registerNewUser` (`REGISTER_URL`). Esa funcion (fuera de este repo) comprueba contra Zoho que el email es cliente de Medicarama.
- Respuestas de registro: **201** creado (se envia email de verificacion), o error con codigo: `not_a_client` (no consta como cliente), `capacity_reached` (saturacion), `weak_password`, `email_already_exists`.
- Tras crear la cuenta se envia verificacion de email y se muestra la pantalla "Revisa tu correo". Un sondeo cada 5 s (`startVerifyPolling`) detecta la verificacion sin que el alumno tenga que pulsar nada.
- Al quedar la sesion verificada se llama a la Cloud Function callable `recordActiveSession` (marca sesion activa, no bloqueante) y se emite el evento `authReady`, que dispara la sincronizacion de memoria con el backend.

`window.tutorAuth` expone `getCurrentToken()` (ID token para las peticiones), `logout()` (borra el historial local y cierra sesion) y `getCurrentUser()`.

7.3. El vinculo alumno -> cursos (capeo)

La Cloud Function `registerNewUser` escribe en Firestore el documento `users/{uid}` con un array `cursos` = `[{nombre, id}, ...]`, donde cada `id` es el `course_id` que el alumno ha comprado en Zoho. Ese array es la "llave" del capeo: define exactamente que material podra consultar. `check_new_user.py` sirve para

verificar que las altas nuevas traen cursos poblado.

8. El capeo por curso (nucleo del sistema)

Implementado en **app/retrieval.py**. Es la pieza que garantiza que un alumno solo acceda al material que ha pagado.

8.1. De uid a course_ids

```
get_user_cursos(uid)      # Firestore users/{uid}.cursos -> [{'nombre','id'}, ...]
get_user_course_ids(uid)  # -> ['3287', '4139'] (solo ids, deduplicados)
```

8.2. Recuperacion filtrada: buscar_contexto

`buscar_contexto(pregunta, course_ids, k=8)` consulta el engine Enterprise medicarama-search con un filtro por curso y devuelve (contexto, fuentes). El filtro es la clausula que hace el capeo:

```
def _filter_expr(course_ids):
    quoted = ",".join(f'"{c}"' for c in course_ids)
    return f"course_id: ANY({quoted})"      # course_id: ANY("3287","4139")

req = de.SearchRequest(
    serving_config=_SERVING,                # engine 'medicarama-search'
    query=pregunta,
    filter=_filter_expr(course_ids),         # <-- EL CAPEO
    page_size=k,
    content_search_spec=CSS(                # extractive answers + snippets
        extractive_content_spec=CSS.ExtractiveContentSpec(max_extractive_answer_count=2),
        snippet_spec=CSS.SnippetSpec(return_snippet=True)))
```

- El filtro `course_id: ANY(...)` solo funciona porque el data store tiene `course_id` marcado como INDEXABLE en su schema (se define al ingestar).
- Cada fuente devuelta lleva `titulo` (nombre del curso o fichero), `url` (URL FIRMADA del PDF), `course_id` y `pagina` (extraida del extractive answer).
- Solo se inyecta el CONTENIDO en el prompt; la fuente se envia aparte para que el modelo no la copie en su respuesta (las fuentes se pintan al final del mensaje, con enlace directo a la pagina del PDF).
- **Fail-closed:** si `course_ids` viene vacio, devuelve ('', []) sin buscar.

8.3. Funciones auxiliares del capeo

Funcion	Que hace
<code>curso_tiene_material(course_id)</code>	True si el corpus tiene al menos 1 documento de ese curso. Cache de 1h. Fail-OPEN (True) ante error: no dar falsos avisos de 'curso no disponible'. Distingue 'curso comprado pero sin material cargado' de 'no encuentre nada'.
<code>pdfs_de_curso(course_id, max_pdfs=12)</code>	Devuelve las URIs <code>gs://</code> de los PDFs del curso, para el modo 'documento completo' (tests/temario, donde los fragmentos no bastan). Respeta el capeo.
<code>firmar_url(gs_uri, minutos=15)</code>	Convierte <code>gs://bucket/obj</code> en URL HTTPS firmada V4. El bucket sigue PRIVADO; firma la service account de Cloud Run via IAM signBlob (sin fichero de clave). Fallback al <code>gs://</code> original si falla.

9. Modelos de IA, grounding y generacion de imagen

9.1. Los tres modelos de texto (app/services.py)

Modelo	Grounding	Uso
model (legacy)	temario + TFG (clasico)	NO usar en paths capados: el grounding clasico mezcla todos los cursos sin filtro (filtra de mas). Se mantiene por compatibilidad.
model_capeo	NINGUNO	Path de texto y PDF. El contexto capado se le INYECTA en el prompt -> imposible que recupere material no comprado. Tambien hace la descripcion por vision de las imagenes generadas.
model_multimodal	solo stores ABIERTOS (TFG + apuntes generales)	Path con archivos. El material de cursos comprados se inyecta (capado); los documentos abiertos llegan por grounding. Nunca el store con capeo.

Los tres son **gemini-2.5-flash** con el mismo `system_instruction`, `filtros_seguridad` y `configuracion_tutor`. La diferencia esencial es que `model_capeo` NO tiene herramientas: esa ausencia ES el capeo.

9.2. Generacion de imagen (migracion de SDK de junio 2026)

El SDK antiguo `vertexai.preview.vision_models.ImageGenerationModel` dejo de resolver Imagen tras su retirada en junio de 2026 (daba 403 not visible to project aunque el modelo existiera). Se migro al SDK nuevo **google-genai**. Verificado: en este proyecto/region el unico Imagen disponible es `imagen-3.0-generate-001`; los demas quedan en cascada por si cambia la disponibilidad.

```
from google import genai
from google.genai import types as genai_types

_MODELOS_IMAGEN = ["imagen-3.0-generate-001", # unico disponible (verificado)
                  "imagen-3.0-generate-002", "imagen-4.0-generate-001"]

def generar_imagen(prompt):
    cli = genai.Client(vertexai=True, project=PROJECT_ID, location=REGION)
    for nombre in _MODELOS_IMAGEN:
        # cascada: usa el 1o que responda
        try:
            r = cli.models.generate_images(model=nombre, prompt=prompt,
                                           config=genai_types.GenerateImagesConfig(number_of_images=1,
                                                                                       aspect_ratio="1:1"))
            if r.generated_images:
                return r.generated_images[0].image.image_bytes
        except Exception as e:
            print(f"Imagen: '{nombre}' no disponible ({e})")
    return None
```

9.3. Contexto por VISION de la imagen generada

Funcion muy valorada: el alumno pide `/img corazon enfermo` y luego pregunta "que enfermedad tiene?". Para que funcione de forma fiable, tras generar la imagen se le pasa la IMAGEN REAL a `model_capeo` (que tiene vision) junto con el material del curso, y su descripcion se guarda en memoria de servidor (`_ultima_imagen[uid]`). En el siguiente turno de texto se inyecta esa descripcion en el prompt (si han pasado menos de 600 s), de modo que la respuesta es concreta sin depender de que el modelo "conecte" el dibujo.

```
# En el handler /img, tras obtener img_bytes:
entrada = [
    Part.from_data(data=img_bytes, mime_type="image/png"), # la imagen REAL
    Part.from_text(desc_prompt), # "MIRA la ilustracion... que enfermedad"
]
resp = model_capeo.generate_content(entrada) # descripcion por vision
_ultima_imagen[uid] = {"desc": nota, "ts": time.time()}
```

Nota: la descripcion se crea MIRANDO la foto (vision), no adivinando por el texto del prompt; es la version mas precisa de esta funcion.

10. Endpoints del backend (app/main.py)

Metodo y ruta	Auth	Descripcion
GET /	No	Sirve index.html (la SPA).
GET /healthz	No	Liveness probe de Cloud Run. Siempre {status: ok}.
POST /chat	Si	Endpoint principal. 3 modos segun entrada (ver 10.1).
POST /generate_pdf_file	Si	Recibe el historial (JSON) y devuelve un PDF de 'apuntes de estudio' reformateando el chat con model_capeo (sin grounding).
POST /update_memory	Si	El frontend sube su historial para re-sincronizar la memoria del backend (sobrescribe chat_history_per_uid[uid]).

10.1. Los tres modos de POST /chat

Entrada: formulario multipart con texto y una lista archivos (0..5). Segun el contenido, el endpoint bifurca:

Modo A - Generar imagen (texto empieza por /img)

- Limpia el prompt, le antepone un estilo ('Ilustracion medica profesional, atlas de anatomia Netter, fondo blanco, educativo...') y llama a generar_imagen (cascada).
- Genera la descripcion por vision (seccion 9.3) y la guarda en _ultima_imagen.
- Devuelve JSON {imagen_base64, nota_historial, fuentes:[]}. La imagen se muestra sola (sin texto que descuadre); nota_historial es una nota OCULTA que da contexto al siguiente turno.

Modo B - Multimodal (hay archivos adjuntos)

- Comprueba el tope diario de tokens.
- Recupera el material capado del texto (buscar_contexto) y lo INYECTA en el prompt; adjunta cada archivo como Part.from_data.
- Llama a model_multimodal.generate_content (bloqueante; los archivos no streamean bien con grounding). Suma los tokens consumidos.
- Devuelve JSON {respuesta, fuentes} donde fuentes = capeo + grounding abierto.

Modo C - Texto (sin archivos) -> streaming NDJSON

Devuelve un StreamingResponse con _stream_text_response(texto, uid). Cada linea es un JSON: {type:chunk,text} (fragmento), {type:done,sources} (fin + fuentes) o {type:error,message}. Flujo interno:

- Comprueba tope de tokens diario; si se supera, responde un mensaje y termina.
- Carga los cursos del alumno. Si no tiene ninguno -> fail-closed (mensaje y fin).
- Construye la query de busqueda con los ULTIMOS turnos del alumno (no solo el ultimo mensaje) para que follow-ups como 'hazme un test de 10' hereden el tema.
- Detecta peticiones 'grandes' (test/examen/quiz/resumen/temario...) -> amplia k a 30 (mas material).
- Si no hay contexto y NINGUN curso comprado tiene material en el corpus, avisa de que el material 'aun se esta cargando' (no un 'no encontrado' confuso).

- **Modo documento completo:** para peticiones grandes, los fragmentos no bastan; se eligen los PDFs del curso dominante (`pdfs_de_curso`) y se pasan ENTEROS al modelo como `Part.from_uri`, con instrucciones de test (opciones A-D, solucion entre `||...||`, y 'continua' si no caben).
- Si no, inyecta el contexto capado + lista de cursos + (si aplica) la nota de la ultima imagen + los ultimos 10 turnos de historial.
- Hace streaming con `model_capeo.generate_content(..., stream=True)`, acumula tokens y emite las fuentes en el done. Ante error, hace rollback del mensaje de usuario.

11. Memoria de conversacion

La memoria del backend es EFIMERA y por usuario: `chat_history_per_uid` es un diccionario en memoria del proceso `{uid: [mensajes]}`. No sobrevive a reinicios y, con varias instancias de Cloud Run, el estado se reparte. Por eso se usan dos mitigaciones:

- **Session-affinity** en Cloud Run: el mismo alumno cae siempre en la misma instancia mientras dura la sesion.
- **Re-sincronizacion desde el frontend**: el navegador guarda el historial completo en `localStorage` (`medicarama_history`) y lo sube por `POST /update_memory` (funcion `syncBackendMemory`) al iniciar sesion, al abrir un chat o al empezar uno nuevo. Asi el backend reconstruye el contexto aunque haya reiniciado.
- `_ultima_imagen` es un diccionario aparte (no lo pisa `/update_memory`) para recordar la ultima imagen generada por usuario durante 10 minutos.

12. Tope de consumo diario por alumno

Para evitar abusos y grandes consumos (sobre todo el modo documento completo, que mete PDFs enteros), hay un tope diario de tokens por usuario, comprobado ANTES de generar (soft-cap). Configurado en `LIMITE_TOKENS_DIA = 400000` (prompt + respuesta, por día UTC).

- `tokens_usados_hoy(uid)`: lee `users/{uid}.tokenDay` y `.tokensUsed` de Firestore; devuelve 0 si el día guardado no es hoy (UTC).
- `sumar_tokens(uid, n)`: suma de forma TRANSACCIONAL (para no perder cuentas con peticiones concurrentes) y reinicia el contador si cambio el día.
- Se cuenta `usage_metadata.total_token_count` de la respuesta de Gemini, en los paths de texto (streaming) y multimodal.
- Al superar el tope, el alumno recibe un mensaje amable ('Vuelve mañana') en vez de un error.

13. Frontend

SPA sin framework. `index.html` es el shell (overlay de login + barra lateral de historial + area de chat + barra de entrada). La logica esta en dos modulos JS.

13.1. `static/js/main.js` (chat)

- **Envio** (`sendMessage`): monta un `FormData` con texto y archivos; si esta el modo imagen, antepone `/img`. Adjunta el token con `authHeaders()`.
- **Streaming** (`handleStreamingResponse`): arquitectura desacoplada de un 'reader loop' (consume el NDJSON lo mas rapido posible) y un 'typewriter pump' (pinta a ~111 caracteres/seg para sensacion de escritura natural). Evita race conditions y permite el boton Stop.
- `processText`: convierte el texto a HTML. Implementa los SPOILERS: el texto entre `||...||` se pinta como un recuadro gris que se revela al hacer click (usado para las soluciones de los tests). Luego pasa por `marked` (markdown) y `KaTeX` (formulas $\$...\$$).
- `renderSources`: filtra por privacidad (oculta fuentes con 'TFG' o 'PRIVADO' en el titulo), deduplica por URL de PDF, muestra maximo 4 y anade el ancla `#page=N` para saltar a la pagina exacta del PDF.
- **Audio (TTS)**: `speakText` con `SpeechSynthesisUtterance` (voz espanola). **Voz (STT)**: `webkitSpeechRecognition` (solo Chrome/Edge; el boton se oculta en navegadores sin soporte).
- **Exportar PDF**: `exportChatToPDF` llama a `/generate_pdf_file` y descarga el blob como `'Apuntes_*.pdf'`.
- **Historial local**: `medicarama_history` en `localStorage`, con limpieza automatica (max 20 chats, 50 mensajes/chat) y estrategia de emergencia si el storage se llena.
- **UX**: modo claro/oscuro persistente, altura real de viewport en movil (`visualViewport -> --app-height` para que el teclado no tape el input), auto-scroll inteligente, lightbox de imagenes, toasts.

13.2. `static/js/auth.js`

Gestiona todo el ciclo de Firebase Auth (seccion 7): login/registro, verificacion de email con sondeo automatico, recuperacion de contrasena, y la exposicion de `window.tutorAuth`. La config de Firebase de cliente y la `REGISTER_URL` estan aqui (son publicas por diseno).

13.3. `index.html`

Contiene el overlay de login con 3 pantallas (login, 'verifica tu correo', 'recuperar contrasena'), la barra lateral, el area de chat, el menu de adjuntar (documentos / imagenes / generar imagen), el lightbox y el footer legal con enlaces a las politicas de Medicarama. Los `?v=FINAL_Vx` en los `<script>`/`<link>` son cache-busting: SUBELOS cuando cambies el JS/CSS o el navegador servira la version cacheada.

14. Seguridad y privacidad

- **Autenticacion obligatoria** (Firebase ID token) y **email verificado** en todos los endpoints sensibles.
- **Capeo fail-closed**: un alumno nunca recupera material de un curso que no ha comprado; sin cursos, no se busca nada.
- **Cabeceras de seguridad** (middleware en todas las respuestas): X-Content-Type-Options: nosniff, X-Frame-Options: DENY (anti-clickjacking), Strict-Transport-Security (HSTS 1 ano), Referrer-Policy, Permissions-Policy.
- **Bucket privado + URLs firmadas V4** (15 min): los PDFs nunca son publicos; se firman con la service account por IAM signBlob.
- **Privacidad de fuentes**: el frontend oculta las fuentes cuyo titulo contiene 'TFG' o 'PRIVADO'.
- **Filtros de seguridad de IA** a `BLOCK\_ONLY\_HIGH` (ajustados al dominio medico, ver seccion 6.2).
- **Anti-abuso**: tope diario de tokens y maximo 5 archivos por mensaje.
- **Secretos**: .env, credentials.json, service-account*.json estan excluidos de git (.gitignore) y del build de Cloud Run (.gcloudignore).

Punto a endurecer: CORS esta configurado con `allow_origins=['*']` (permisivo). Como la API exige Bearer token valido, el riesgo es limitado, pero conviene restringirlo al dominio de produccion cuando se establezca.

15. Evaluacion empirica (OE6)

La carpeta `evaluacion/` contiene un harness (`harness_eval.py`) y cuatro baterias de preguntas para medir el sistema de forma reproducible. Resultados de la ultima ejecucion:

Bateria (n)	Que mide	Resultado
Recuperacion (150)	Cobertura y acierto del RAG capado	Cobertura 93.3% (≥ 1 fuente); Hit@k 92.7%; latencia recuperacion ~ 0.26 s.
Aislamiento (20)	Fugas entre cursos (adversario)	0 fugas / 100% aislamiento correcto: nunca devuelve material de otro curso.
Anti-alucinacion (20)	Preguntas fuera del corpus	Declina 18/20 (90%): reconoce que no tiene la informacion.
Rechazo clinico (12)	Peticiones de consejo clinico inseguro	Declina 10/12 (83.3%): deriva a profesional en vez de prescribir.

El harness mide Hit@k comparando el documento esperado contra titulo + url de las fuentes. **Aviso:** en `evaluacion/_run1.log` hay una corrida ANTIGUA sobre un subconjunto (36 preguntas) con Hit@k 19.4%; fue un ARTEFACTO de una version que solo comparaba contra el titulo (nombre del curso). Los CSV de resultados actuales reflejan las cifras correctas de la tabla. Las heurísticas de 'declinar' (marcadores tipo 'no consta', 'consulta a un profesional') tienen columnas para revision manual.

```
# Ejecutar la bateria completa (requiere ADC valido):
python3 evaluacion/harness_eval.py \
  --preguntas evaluacion/preguntas.csv --gen \
  --aislamiento evaluacion/preguntas_aislamiento.csv \
  --fora-corpus evaluacion/preguntas_fora_corpus.csv \
  --refus-clinic evaluacion/preguntas_refus_clinic.csv
```

16. Tests y utilidades

Script	Que hace
test_capeo_e2e.py	Smoke test end-to-end del capeo de texto SIN levantar uvicorn: busca un usuario real con cursos (o siembra uno temporal 3287/4139), y ejecuta <code>get_user_course_ids</code> -> <code>buscar_contexto</code> -> <code>_stream_text_response</code> completo.
test_multimodal_capeo.py	Smoke test del path multimodal: material capado inyectado + imagen, con <code>model_multimodal</code> (grounding solo de stores abiertos).
check_new_user.py	Lista usuarios por fecha de alta y muestra si traen 'cursos' desde Zoho. Util para verificar que el registro Zoho -> Firestore funciona.
app/retrieval.py (main)	Ejecutado directamente hace un smoke test de la busqueda RAG filtrada: <code>python3 -m app.retrieval 'consulta' 3287 4139</code> .

17. Despliegue (Cloud Run)

El contenedor se define en el **Dockerfile** (base python:3.10-slim, instala requirements, copia app/, static/ e index.html, y arranca uvicorn app.main:app --host 0.0.0.0 --port 8080).

17.1. Comando de despliegue

```
gcloud run deploy unibot-medicarama \  
  --source /Users/nestorsoriano/unibot/tutor-unis \  
  --region us-central1 --project tutor-unis  
# Cloud Build construye la imagen desde el código (respetando .gcloudignore)  
# y la publica en Cloud Run.
```

17.2. Configuración recomendada del servicio

- **Memoria:** 1Gi (los PDFs enteros del modo documento completo consumen).
- **min-instances 1:** evita arranques en frío (el primer request pagaba la inicialización de Vertex AI / Firebase).
- **Session-affinity activado:** necesario para la memoria en proceso (sección 11).
- **Service account** del servicio con permisos: Vertex AI User, Discovery Engine (search), Firestore, Storage Object Viewer y **Service Account Token Creator sobre sí misma** (imprescindible para firmar las URLs sin archivo de clave).

17.3. .gcloudignore

Excluye del build lo que no debe ir al contenedor: .venv/, .git/, .env y otros secretos, node_modules/, tests/ y test_*.py, docs/, *.pdf, *.md y README.txt. Por eso ESTA documentación (docs/, *.pdf) va a GitHub pero NO engorda la imagen de Cloud Run.

18. Operativa y resolucion de problemas

Sintoma	Causa y solucion
503 'Reauthentication is needed' en local	ADC caducado (política de la org mindic.com). Ejecuta de nuevo <code>gcloud auth application-default login</code> . No es un bug.
La generacion de imagenes falla (403 'not visible' o 'ningun modelo')	Disponibilidad de Imagen. Hoy solo 'imagen-3.0-generate-001' esta visible en el proyecto. Verifica con el SDK google-genai (seccion 9.2).
Un alumno dice que 'no ve su curso'	Ejecuta <code>check_new_user.py</code> y confirma que su <code>users/{uid}.cursos</code> trae el <code>course_id</code> . Si el curso no tiene material aun, el sistema lo avisa solo.
Las fuentes no abren / dan error de permisos	URL firmada caducada (15 min) o falta el rol Token Creator en la SA. Revisa <code>firmar_url</code> y los permisos de la service account.
El frontend no refleja cambios de JS/CSS	Cache del navegador. Sube el sufijo <code>?v=FINAL_Vx</code> en <code>index.html</code> .

19. Estado del repositorio y deuda tecnica

- **Git desactualizado:** el repositorio solo tiene 2 commits (ultimo de enero 2026). Casi todo el trabajo reciente (capeo, tokens, imagen, evaluacion) esta SIN commitear. Primer paso al retomar: revisar `git status` y hacer commits ordenados.
- **.pyc en git:** hay ficheros `__pycache__/* .pyc` trackeados de antes de crear el `.gitignore`. Conviene `git rm -r --cached app/__pycache__` para dejar de versionarlos (el `.gitignore` ya los excluye a futuro).
- **OPEN_DATA_STORE_ID sin configurar:** la funcion de 'documentos abiertos' (apuntes generales para todos los alumnos) esta implementada pero dormida hasta que se cree ese data store y se ponga la variable.
- **Modelo legacy** (`model + herramienta_temario`): mezcla todos los cursos sin filtro; se mantiene solo por compatibilidad y NO debe usarse en paths capados.
- **Memoria en proceso:** se pierde en reinicios y no se comparte entre instancias (mitigado con `session-affinity + re-sync` del front). Para escalar de verdad convendria una memoria compartida (p. ej. Firestore o Redis).
- **CORS permisivo (*):** restringir al dominio de produccion.
- **Disponibilidad de Imagen fragil:** depende de un unico modelo visible; la cascada ayuda pero conviene vigilar retiradas futuras de Google.

20. Glosario

Termino	Significado
Capeo	Filtrado del material por los cursos que el alumno ha comprado (course_id). Nucleo del sistema.
RAG	Retrieval-Augmented Generation: recuperar contexto y darselo al modelo para que responda basandose en el.
Grounding	Herramienta de Vertex que deja al modelo buscar por su cuenta en un data store. En los paths capados se EVITA (se inyecta el contexto).
Fail-closed	Ante la duda o falta de datos, el sistema NO muestra material (cierra en vez de abrir).
Data store / Engine	Almacen indexado de documentos en Vertex AI Search / aplicacion de busqueda por encima (Enterprise para extractive answers).
Extractive answer	Fragmento textual que Vertex extrae del documento como respuesta directa; se usa como contexto y da el numero de pagina.
ADC	Application Default Credentials: credenciales locales de gcloud que usa el codigo para autenticarse contra GCP.
NDJSON	Newline-delimited JSON: un objeto JSON por linea; formato del streaming de /chat.
Session-affinity	Cloud Run enruta al mismo alumno a la misma instancia mientras dura la sesion.

Fin del documento. Para dudas sobre decisiones concretas, el codigo esta ampliamente comentado en espanol, sobre todo en `app/retrieval.py`, `app/services.py` y `app/main.py`.